TransZKEval: Unraveling LLM Behaviors in ZKP Circuit Translation

***Abstract*—Zero-knowledge proofs (ZKPs) are critical to scalable and privacy-preserving blockchains, yet manually translating imperative programming languages (IPLs) into circuits such as Circom remains labor-intensive and vulnerable to under-constrained bugs. While large language models (LLMs) promise automated IPL-to-ZKP circuit translation, their capability remains under-explored due to the lack of dedicated benchmarks. To bridge this gap, we present *TransZKEval*, the first benchmark for evaluating IPL-to-circuit translation. It includes 100 method-level tasks across 8 domains and 4 mainstream languages including Python, Java, Rust, and Go, supported by 400 manually verified Circom implementations and high-coverage test suites.**

We select seven representative LLMs and generate a total of 19,600 samples via *TransZKEval*. Using these samples, we conduct extensive empirical analyses to explore translation correctness, translation strategies, circuit security, and typical failure patterns. Results show that even top-tier LLMs achieve only 2%-24% computational accuracy, representing a drastic decline compared to conventional code translation. Domain-specific rule injection and error-aware iterative refinement significantly boost performance, while intermediate representations and generic retrieval often cause negative transfer. Security analysis further reveals that many functionally correct circuits still contain under-constrained vulnerabilities. Our in-depth 9-category, 81-subcategory error taxonomy, constructed from 47,765 failed cases, identifies undefined symbols, syntax confusion, and constraint violations as the dominant failure modes. Based on these findings, we derive an error-guided refinement strategy from these findings, which yields performance comparable to expert-designed rules. *TransZKEval* and our results provide valuable insights for future research on reliable and secure circuit synthesis.

***Index Terms*—Zero-Knowledge Proof, ZKP Circuit, Circom, Code Translation, Large Language Model, Benchmark**

I. Introduction

Zero-Knowledge Proofs (ZKPs), particularly zk-SNARKs, are essential for blockchain scalability and privacy [1], [2]. Deploying ZKP-based applications requires migrating existing application logic into ZKP circuits. Nevertheless, many of these components, such as DeFi protocols, are already implemented in imperative programming languages (IPLs) like Rust, Python, Go or Java [3]–[5], and these PLs lack native compatibility with ZKP circuit representations.

To deploy such logic in ZKP systems, developers rewrite imperative implementations as circuits using domain-specific languages (DSLs). Among existing DSLs, Circom has become the prevailing choice due to its mature toolchains and high proving efficiency [6]. However, because no automated translation toolchain exists from IPLs to Circom, this rewriting process must be performed manually, and it has become the common practice in real-world ZKP development [7].

This manual process is inherently difficult. As illustrated in Figure 1, simple operations like `a ^ b` in Rust need to be rebuilt into polynomial constraints in Circom. Circom lacks native boolean support, requiring constraints such as $a \cdot (1 - a) = 0$ and $res = a + b - 2ab$ to conform to R1CS rules. Such algebraic transformation is time-consuming and error-prone, easily leading to under-constrained vulnerabilities. Given these difficulties, an automated approach is highly desirable. In particular, this task is inherently a code translation problem, and LLMs have proven highly effective in this domain [8], [9], making them a natural candidate for automating IPL-to-circuit translation. Despite this potential, the effectiveness of LLMs in porting IPLs to ZKP constraints remains underexplored, and no existing benchmark supports IPL-to-ZKP Circuit translation.

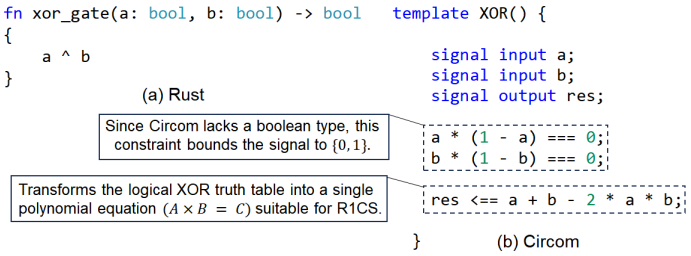

Fig. 1. Example: XOR gate in Rust vs. Circom

To bridge this gap, we propose *TransZKEval*, the first benchmark for IPL-to-ZKP circuit translation. It contains 100 method-level tasks in Python, Java, Rust and Go, with 400 verified Circom implementations spanning eight domains including DeFi and cryptography. All cases are manually reviewed by ZKP experts to ensure functional equivalence. We further generate 19,600 LLM-produced circuit samples for empirical evaluation. Adopting standard test quality metrics [10]–[12], our test suites reach 99.78% statement coverage and 82.22% mutation score, which provides strong evidence of semantic equivalence between source and target implementations.

This study uses *TransZKEval* to systematically evaluate 7 representative LLMs on IPL-to-ZKP circuit translation. We propose 4 research questions (RQ1–RQ4) to investigate: RQ1 focuses on overall translation correctness, RQ2 on the effectiveness of translation strategies, RQ3 on circuit security, and RQ4 on typical generation error patterns. Experiments show IPL-to-ZKP circuit translation is highly challenging, even top-tier LLMs only achieve 2%-24% computational accuracy, a significant drop compared to their performance on conventional IPL-to-IPL translation. Domain-specific rule injection

and iterative refinement improve performance, while intermediate representations and generic retrieval cause negative transfer. Security analysis reveals many functionally correct circuits still contain underconstrained vulnerabilities. By analyzing 47,765 failed cases, we establish a 9-category, 81-subcategory error taxonomy, identifying key issues like undefined components and constraint violations. We then present an error-aware refinement strategy, whose performance runs neck-to-neck with manually designed expert rules. These findings provide practical guidance for developers, helping them select suitable LLMs, adopt effective translation strategies, and avoid common errors in IPL-to-ZKP circuit translation.

Overall, this work makes three key contributions:

(1) We present *TransZKEval*, the first dedicated benchmark for evaluating IPL-to-ZKP-circuit translation, covering diverse domains and rigorous quality assurance. The benchmark dataset and associated code are available at [13].

(2) We conduct the first large-scale empirical study of LLMs on IPL-to-ZKP circuit translation, revealing performance gaps, strategy efficacy, security risks, and fine-grained error patterns.

(3) We construct a 9-category, 81-subcategory error taxonomy derived from 47,765 failed cases, with illustrative examples and repair guidelines, and validate that knowledge injection and error-aware iterative refinement effectively boost translation performance, yielding actionable guidance for building more robust LLM-based ZKP toolchains.

We believe *TransZKEval* and our findings will facilitate future research toward more reliable, secure, and automated ZKP circuit development.

II. New Benchmark: *TransZKEval*

This section introduces *TransZKEval*, a dedicated benchmark for evaluating LLMs on translating IPLs into Circom. We elaborate the benchmark design, task taxonomy, construction pipeline, and evaluation metrics for quality assurance.

A. Source Code Construction

We collect 100 tasks covering four mainstream IPLs: Python, Java, Rust, and Go. All tasks are categorized into eight ZKP-relevant domains, which are designed according to the core functional modules and typical engineering scenarios of ZKP systems to ensure comprehensive coverage of representative and frequently encountered programming cases, as summarized in Table I. We adopt method-level granularity to isolate external dependencies, focusing evaluation purely on the logical conversion capability of LLMs.

TABLE I
Taxonomy and domain definitions of the 100 tasks in *TransZKEval*.

Category	Definition	# Tasks	# Files	# TC	# GS
Simple Control-Flow	Basic arithmetic & bitwise logic.	20	100	100	3,920
Artificial Intelligence	Neural network inference kernels.	15	75	75	2,940
DeFi Applications	Automated markets & asset vaults.	15	75	75	2,940
Games & Interaction	Verifiable state transitions & fairness.	10	50	51	1,960
Cryptography	Core primitives & hashing algorithms.	10	50	54	1,960
Blockchain	Ledger data model verification.	10	50	53	1,960
Data Structures	Standardized ADT manipulation.	10	50	50	1,960
Statistics	Numerical analysis & data aggregation.	10	50	53	1,960
Total		**100**	**500**	**511**	**19,600**

Note: # denotes count; TC = Test Cases; GS = Generated Samples.

We follow three standardized principles to ensure cross-language consistency: **(1) Unified Functional Requirements.** Each task is defined with identical algorithmic logic, and implemented independently across four IPLs to preserve consistent computational semantics while adhering to language-specific coding styles. **(2) Coding and Naming Conventions.** We follow official style guides (e.g., PEP 8 [14], Google Java Style Guide [15]) and consistent naming rules. All tasks are encapsulated in single files with comments removed to avoid providing extra hints for LLMs. **(3) Unified Interface Constraints.** We restrict input and output to integer types, facilitating cross-PL result comparison and compatibility with finite-field elements in ZKP circuits.

After construction, we verify cross-language semantic equivalence through automated testing and expert review. We adopt language-agnostic input-output pairs to unify expected outputs, with language-specific test scripts for practical execution. We take Python as the representative language to assess test sufficiency using standard testing metrics [10]–[12]. We adopt `pytest-cov` to compute statement and branch coverage, and `mutmut` to conduct mutation testing. Higher metric values imply more adequate test scenarios. Our test suites reach 99.78% statement coverage, 99.47% branch coverage and 82.22% mutation score, surpassing the 80% standard for robust test suites [16].

Two software engineers with $\geq$ 3 years of experience then cross-review all implementations to validate adherence to unified requirements and consistent edge-case handling. This process yields 400 verified source samples (100 tasks $\times$ 4 PLs) and required approximately 100 person-hours.

B. Ground Truth and Test Suite Construction

To establish a gold standard for *TransZKEval*, we conduct a rigorous process to develop the Circom reference implementations and a corresponding automated validation infrastructure.

Expert Circuit Labeling. Two ZKP domain experts with $\geq$ 3 years of experience manually translated the verified imperative logic into Circom, re-implementing algorithms using arithmetic gates and signals while handling signal dependencies and finite field arithmetic. Each circuit underwent peer review, with a second expert independently verifying semantic alignment between the imperative source and target circuit.

Automated Verification and Evaluation Pipeline. We built a unified automated pipeline for both ground truth validation and LLM evaluation, running within a process-level sandbox and its core logic is formalized in Algorithm 1. The pipeline first sanitizes redundant headers and injects a main component template matching the task signature S if absent. It then compiles the circuit via Circom to generate R1CS metadata and a Wasm witness generator, extracting the constraint count $N_{constraints}$ as a complexity metric. A dynamic signal mapping mechanism bridges imperative test vectors and circuits by auto-populating `input.json`. The pipeline executes the witness generator and extracts outputs from `w.json`, enforcing functional equivalence via $actual \equiv expected \pmod P$ across all test cases.

Algorithm 1: Evaluation Pipeline for TransZKEval

Data: Circom code C, Task signature S, Test suite T, Scalar field P
Result: Evaluation Report (Status, Constraints, Pass Rate)

$C' \leftarrow \text{Sanitize}(C)$;
if *HasMain*$(C') =$ *false* **then**
  $C_{full} \leftarrow \text{InjectMain}(C', S)$;
else
  $C_{full} \leftarrow C'$;
end
$Res_{comp} \leftarrow \text{CircomCompile}(C_{full})$;
if Res_{comp} *fails* **then**
  return COMPILE_FAIL
end
$N_{constraints} \leftarrow \text{ExtractConstraints}(Res_{comp}.r1cs)$;
$Signals \leftarrow \text{ParseInputSignals}(C_{full})$;
for *each* $(input, expected) \in T$ **do**
  $InputJSON \leftarrow \text{MapSignals}(Signals, input)$;
  $Witness \leftarrow$ GenWitness$(Res_{comp}.wasm, InputJSON)$;
  $Actual \leftarrow \text{ExtractOutput}(Witness)$;
  if $Actual \not\equiv expected \pmod P$ **then**
    return LOGIC_FAIL
  end
end
return SUCCESS, $N_{constraints}$;

The entire labeling and validation process consumed approximately 200 person-hours, resulting in a high-fidelity parallel corpus with strictly verified functional equivalence.

III. Experimental Design

This study adopts *TransZKEval* to evaluate LLM performance on translating imperative code into ZKP circuits, with four core research questions clarified as follows.

RQ1 (Overall Correctness) We explore the baseline performance of LLMs in this cross-paradigm translation task. Results reveal that LLMs struggle heavily with converting program logic into formal mathematical constraints, showing far lower accuracy than general code translation tasks. This finding clarifies the inherent difficulty of ZKP-oriented code generation for practitioners.

RQ2 (Strategy Effectiveness) We analyze how different translation strategies influence model output quality. Experimental results verify that domain knowledge injection and iterative refinement bring prominent gains, while IR-based methods and naive retrieval easily cause negative transfer. The conclusions enable developers to select efficient enhancement strategies reasonably.

RQ3 (Security and Soundness) We examine the security and constraint completeness of generated circuits. We find plenty of functionally correct circuits still contain under-constrained loopholes that threaten practical usage. This reminds developers to attach equal importance to functional correctness and constraint sufficiency in circuit deployment.

RQ4 (Error Analysis) We sort out common error categories and their distribution rules in circuit translation, and summarize typical flaws of LLMs in this task. These findings help developers quickly identify and rectify frequent translation errors in practical development. Guided by these error characteristics, we propose an error-aware iterative refinement strategy. This optimization approach helps developers improve translation quality and reduce manual revision costs.

TABLE II
Key Specifications of the Studied LLMs

Category	Model	Release Time	Size	Training Base	In/Out (Tokens)
Commercial	GPT-5.4	2026-03	–	–	1M / 128k
	Gemini 3.1 Pro	2026-02	–	–	2M / 128k
	DeepSeek-V3.2	2026-01	685B	14.8 trillion	128k / 4k
	Qwen-Max	2026-01	1T	36 trillion	258048 / 32768
Code	Qwen3-Coder-Plus	2025-09	480B	7.5 trillion	997952 / 65536
General	Llama 3.1	2024-07	8B	15 trillion	128k / 4k
	Llama 3.1	2024-07	70B	15 trillion	128k / 4k

A. Studied LLMs

Table II lists our studied LLMs along with their categories, released times, and technical specifications. To extensively explore the performance of LLMs in the specialized domain of IPL-to-ZKP circuit translation, we select a representative set of LLMs across three categories. First, we include top-tier commercial LLMs, namely GPT 5.4 [17], Gemini 3.1 Pro [18], Qwen-max [19] and DeepSeek-V3.2 [20], to evaluate the upper bound of neural code intelligence in complex logic synthesis. Second, we select Qwen3-Coder-Plus [21] as a representative of specialized code-centric LLMs to investigate the advantages of domain-specific pre-training. Finally, we include Llama 3.1 (8B and 70B) [22] to study the influence of model scale and the capabilities of SOTA open-source architectures.

B. Studied Translation Strategies

To comprehensively evaluate how LLMs bridge the gap between imperative logic and arithmetic circuits, we investigate six translation strategies across four distinct categories. The detailed prompt templates for these strategies follow the experimental settings established in related code intelligence research [23] and are openly available in our repository [13].

(1) Direct Translation (Direct): This strategy serves as the baseline, where LLMs are provided with the raw source-language snippet and asked to generate the target Circom code in an end-to-end, zero-shot manner. It measures the LLM's inherent capacity for IPL-to-ZKP Circuit translation without any auxiliary context or structural guidance.

(2) Intermediate Representation-Guided Translation (IR): Following existing research [24], [25], we adopt a two-stage pipeline. We first convert source imperative code into unified IRs, and then map such representations to standard Circom circuits, so as to decouple core logic from original language syntax. This category investigates three distinct IR variants: *CoT* (natural language reasoning steps), *Pseudocode* (structured, language-agnostic logic), and *NL* (high-level functional descriptions). These IRs serve as a semantic bridge, guiding the LLM to focus on logic decomposition before generating the final Circom constraints.

(3) Knowledge-Injected Translation (KI): Recognizing that IPL-to-ZKP Circuit translation requires domain-specific expertise, we investigate two knowledge injection methods to equip LLMs with domain-specific practical knowledge:

- *Rule-Injected Translation (Rule)*: This approach explicitly injects translation prompts and deterministic transformation rules manually summarized by experts responsible for constructing TransZKEval, acting as a manual guide for complex logic transformation.
- *Retrieval-Augmented Translation (RAG)*: We retrieve similar implementation guidance or idiomatic Circom templates from a curated vector database that is constructed by crawling two sources in parallel: (1) the top 100 GitHub projects (ranked by stars) returned for the keyword "Circom", splitting each project by template and covering diverse categories such as core circuits, libraries, privacy protocols, machine learning, and interoperability frameworks (the complete crawled data available in our repository), and (2) the official Circom documentation.

(4) Iterative Refinement (Refinement): To simulate the realistic debugging process, this strategy allows the model to refine its output based on feedback. If the initial translation fails compilation or functional verification, the error messages (e.g., compiler logs or test failure reports) are fed back into the LLM as a prompt for self-correction and iterative optimization.

C. Evaluation Metrics

We follow previous studies [23], [26], [27] and evaluate LLMs via Compilation Success Rate (CSR), Pass Rate (PR), and Computational Accuracy (CA). Additionally, we propose Difference in Insecure Rate (DIR) as a new metric to evaluate LLMs' capability in generating secure ZKP circuits.

Compilation Success Rate (CSR) [8], [23]: It computes the ratio of samples that can be successfully compiled after translation. We define CSR as below.

$$CSR = \frac{\sum_{k=1}^{N} cs(\hat{y_k})}{N}, \text{ where } cs(\hat{y_k}) = \begin{cases} 1 & Compile(\hat{y_k}) \rightarrow success \\ 0 & Compile(\hat{y_k}) \rightarrow error \end{cases} \tag{1}$$

where N denotes the number of samples, $\hat{y_k}$ denotes the k-th translated sample via a LLM. *Compile(·)* denotes compiling samples with their corresponding compilers (e.g., the *Circom* compiler), and its results are either *success* or *error*.

Pass Rate (PR) [8], [23]: It computes the ratio of successfully passed test cases across all samples, where a passed test case means the execution result of a translated program equals that of the ground truth program. We define PR as below.

$$PR = \frac{\sum_{k=1}^{N} \sum_{j=1}^{T_k} ps(y_k, y_k, j)}{\sum_{k=1}^{N} T_k}, \text{where } ps(\hat{y_k}, y_k, j) = \begin{cases} 1 & Exec_{k,j}(y_k, j) = Exec_{k,j}(\hat{y_k}, j) \\ 0 & Exec_{k,j}(y_k, j) \neq Exec_{k,j}(\hat{y_k}, j) \end{cases} \tag{2}$$

where T_k is the total number of test cases for the k-th sample, and y_k denotes the ground truth of the k-th sample. $Exec_{k,j}(\cdot)$ denotes the execution result of a program (y_k or $\hat{y_k}$) on the j-th test case from the test suite of the k-th sample.

Computational Accuracy (CA) [8], [23]: It computes the ratio that the translated programs can produce the same execution result as the ground truths, given a set of test suites. We define CA as below.

$$CA = \frac{\sum_{k=1}^{N} ca(y_k, \hat{y_k})}{N}, \text{ where } ca(y_k, \hat{y_k}) = \begin{cases} 1 & Exec_k(y_k) = Exec_k(\hat{y_k}) \\ 0 & Exec_k(y_k) \neq Exec_k(\hat{y_k}) \end{cases} \tag{3}$$

where $Exec_k(\cdot)$ denotes the execution result of a program with the test suite of the k-th sample.

Diff in Insecure Rate (DIR): We propose Diff in Insecure Rate (DIR) as the core evaluation metric to quantify security discrepancies among ZK circuit generation LLMs and strategies. A circuit is deemed insecure when suffering from under-constraint issues, namely logical loopholes that allow invalid proof verification. DIR quantifies the insecure rate gap between two LLMs over their mutually processable circuit samples. Its formal definition is:

$$DIR_{A,B} = UR_{A|I_{AB}} - UR_{B|I_{AB}} \tag{4}$$

where the insecure rate $UR = \frac{N^t_{insecure}}{N}$, $N^t_{insecure}$ denotes the number of circuits classified as insecure by the analyzer t, and N is the total number of samples. I_{AB} represents the intersection of circuits successfully processed by both LLM A and LLM B, and $UR_{A|I_{AB}}$ is the insecure rate of LLM A evaluated exclusively on this common sample set.

A positive DIR indicates that LLM A is less secure than LLM B, while a negative DIR indicates the opposite. This metric eliminates misleading evaluation results, where an inferior LLM might appear more secure merely by handling only simpler, less error-prone circuits.

D. Implementation Details

We evaluate *TransZKEval* using 7 state-of-the-art LLMs, including DeepSeek-V3.2, Qwen-Max, Qwen3-Coder-Plus GPT-5.4, and Gemini-3.1-pro accessed via official APIs [17], [18], [28], [29], as well as Llama3.1-8B, and Llama3.1-70B operated through the NVIDIA platform [30]. To ensure reproducibility and minimize stochasticity, we set $temperature = 0$ and $n = 1$, only evaluating the first generated candidate for each task in a one-shot setting. For circuit analysis and security verification, we incorporate specialized tools including CONSCS [31], ZKFuzz [32], and Picus [33]. The experiments related to verification are conducted on a high-performance server equipped with an Intel Xeon Gold 6226R CPU server with 503 GB RAM, running Ubuntu 18.04.6 LTS. Following prior studies [24], [34], we assessed experimental stability by repeating a representative subset of translations 3 times, which revealed a negligible PR variance (mostly $< 3\%$ for DeepSeek-V3.2), confirming that inherent LLM randomness has no significant influence on our conclusions.

IV. Evaluation Results

A. RQ1: Overall Correctness

To understand the challenge of ZK circuit translation and its divergence from IPL-to-IPL translation, we evaluate each LLM on the *TransZKEval* benchmark, focusing on end-to-end translation from IPLs (Rust, Java, Python, Go) to Circom. Figure 2 compares model performance (CA, CSR, PR) against imperative-to-C++ translation, while Table III details

results across translation pairs using the Direct strategy. Key observations are as follows.

1) IPL-to-ZKP Circuit vs. IPL-to-IPL Translation: As shown in Figure 2, while prior IPL-to-IPL translation studies [9], [23], [35] and our own C++ baseline both achieved remarkably high accuracy, LLM performance drops sharply when shifting from standard IPL-to-IPL translation to IPL-to-ZKP Circuit translation. For the C++ baseline, the studied LLMs achieve near-perfect scores, averaging 99.18% in CSR, 96.28% in PR, and 94.32% in CA, confirming that state-of-the-art LLMs are well-equipped for conventional code migration tasks where source and target languages share similar imperative paradigms and control-flow structures. In stark contrast, their performance on *TransZKEval* suffers a catastrophic decline, with average CSR, PR, and CA plummeting to 15.18%, 13.73%, and 12.82%, respectively. This corresponds to a performance degradation of 84.70%, 85.74%, and 86.41%. This dramatic gap highlights that IPL-to-ZKP Circuit translation is far more challenging than IPL-to-IPL translation, as it requires decompiling imperative logic into mathematical constraint system over finite fields, a concept with no direct equivalent in IPLs.

Finding 1: Compared with prior IPL-to-IPL translation studies and our own experiment, which both reported high accuracy, LLMs manifest a drastic performance decline when tasked with IPL-to-ZKP Circuit translation. This highlights that the shift from imperative execution logic to constraint-based mathematical verification poses a fundamental barrier for current LLMs.

2) Comparison among LLMs: Focusing on the results within the *TransZKEval* benchmark (see Table III), commercial and high-tier LLMs demonstrate a decisive advantage. These LLMs achieve average scores of 19.13% in CSR, 17.55% in PR, and 16.56% in CA, while smaller LLMs lag behind with scores of 9.92% in CSR, 8.64% in PR, and 7.83% in CA. This comprehensive performance gap highlights the collective impact of massive parameter scales and sophisticated proprietary architectures. Specifically, the superiority of commercial LLMs like Gemini-3.1-Pro and GPT-5.4 is likely due to their advanced reasoning capabilities and extensive training on high-quality proprietary code corpora.

Within the open-source community, the effectiveness of scaling laws remains evident as Llama-3.1-70B substantially outperforms Llama-3.1-8B across all metrics, with scores of 9.50% in CSR, 8.17% in PR, and 7.50% in CA compared to 4.00% in CSR, 3.47% in PR, and 2.50% in CA. Furthermore, domain specialization plays a critical role in this niche field. The specialized Qwen-Coder exhibits a relative lead over the general-purpose Qwen-Max, surpassing it with scores of 16.25% in CSR, 14.29% in PR, and 13.50% in CA against 12.25% in CSR, 11.16% in PR, and 10.50% in CA. This result indicates that code-specific pre-training provides a superior semantic foundation for IPL-to-ZKP Circuit translation compared to general-purpose scaling alone. Overall, the results suggest that tackling the unique challenges of *TransZKEval* requires a combination of high-capacity model architectures and extensive exposure to diverse PLs.

TABLE III
DETAILED PERFORMANCE ACROSS DIFFERENT SOURCE PLS/LLMS IN *TransZKEval*.

Model	Go			Java			Python			Rust		
	CSR	PR	CA	CSR	PR	CA	CSR	PR	CA	CSR	PR	CA
Gemini-3.1-Pro	21.00	20.35	20.00	**24.00**	**23.29**	**23.00**	**20.00**	**19.57**	19.00	**26.00**	**24.85**	**24.00**
GPT-5.4	**25.00**	**24.07**	**23.00**	21.00	20.35	19.00	**20.00**	18.79	**19.00**	19.00	17.81	17.00
DeepSeek-V3.2	**25.00**	20.74	18.00	21.00	15.66	13.00	15.00	14.09	14.00	20.00	16.63	14.00
Qwen-Coder	15.00	13.50	12.00	17.00	14.68	14.00	19.00	16.24	15.00	14.00	12.72	13.00
Qwen-Max	12.00	10.76	10.00	10.00	9.59	9.00	15.00	13.31	13.00	12.00	10.96	10.00
Llama-3.1-70B	10.00	8.81	8.00	8.00	7.83	8.00	8.00	5.68	4.00	12.00	10.37	10.00
Llama-3.1-8B	3.00	2.74	2.00	3.00	2.54	2.00	6.00	4.89	3.00	4.00	3.72	3.00
Avg.	15.86	14.42	13.29	14.86	13.42	12.57	14.71	13.22	12.43	15.29	13.87	13.00

Note: For each column, the best performance is highlighted in **bold**.

Finding 2: Commercial LLMs consistently dominate due to their model scale and superior proprietary architectures. Meanwhile, larger open-source variants outperform smaller ones, and code-specific LLMs further leverage specialized training to surpass general-purpose counterparts in the ZK translation task. This aligns with earlier findings from IPL-to-IPL code translation research, providing programmers with practical insights for LLM selection.

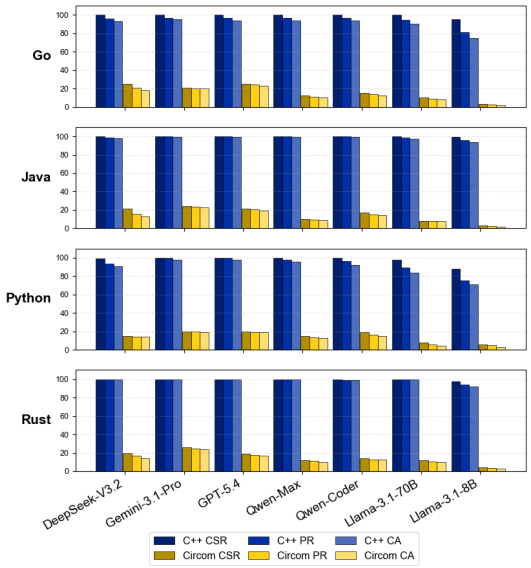

Fig. 2. Performance gap between IPL-to-IPL (C++) translation and IPL-to-ZKP Circuit translation among Diverse LLMs

3) Comparison among translation directions: Based on program similarity analysis (Table IV) and Figure 3, we observe a distinct shift in the role of PL proximity. For the C++ baseline, higher similarity scores correlate with higher success rates, indicating that IPL-to-IPL translation is an incremental migration where source syntax serves as a template. In contrast, this correlation collapses in the Circom translation task. Despite variations in similarity across source PLs, these differences yield no performance gains, revealing that IPL-to-ZKP Circuit translation translation is a reconstructive arithmetization process rather than a syntactic mapping. Consequently, source IPL similarity ceases to be a meaningful predictor of success. Unlike prior IPL-to-IPL studies and our baseline, where PL proximity drove high accuracy, the IPL-to-ZKP Circuit translation task nullifies this advantage, underscoring

that evaluating LLMs for ZK development should prioritize constraint reasoning over syntactic resemblance.

Finding 3: While prior IPL-to-IPL translation studies show that LLMs benefit from syntactic and structural similarities between closer language pairs, our findings reveal that this advantage disappears when targeting Circom. Unlike code-pattern reuse in IPL translation, converting logic to Circom requires reconstructing programs as arithmetic constraints, where surface-level resemblance offers no benefit. Compared with previous conclusions, this contrast underscores that developers should prioritize a model's constraint-reasoning capability over its performance on syntactically similar IPL pairs when selecting LLMs for zero-knowledge circuit development.

TABLE IV
DETAILED LEXICAL AND SEMANTIC SIMILARITY ACROSS SOURCE IPL.

Target PL	Python		Go		Rust		Java	
	Jac.	Cos.	Jac.	Cos.	Jac.	Cos.	Jac.	Cos.
C++	0.51	0.75	0.63	0.66	0.62	0.79	0.73	0.79
Circom	0.24	0.39	0.29	0.47	0.26	0.45	0.27	0.44

Note: Jac. denotes Jaccard similarity [36] for lexical token overlap, and Cos. denotes cosine similarity [37] for semantic embedding similarity.

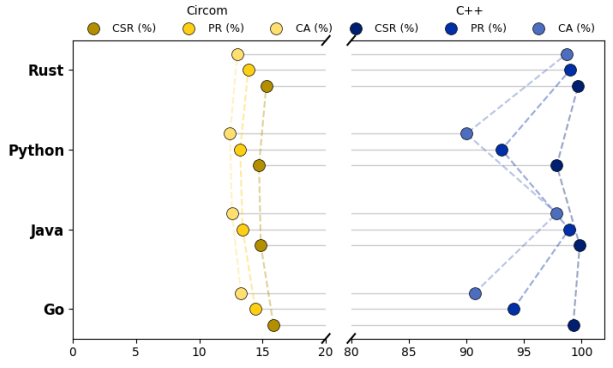

Fig. 3. Relative Performance of Different Source IPLs on IPL-to-ZKP Circuit and IPL-to-IPL Translation

B. RQ2: Translation Strategies

To investigate whether the strategy preferences observed in prior IPL-to-IPL translation studies still hold for IPL-to-ZKP circuit translation, and to identify which strategies are best suited for this more demanding IPL-to-ZKP Circuit translation task, our studied ZKP-oriented code translation strategies can be divided into three categories, i.e., direct translation, IR-based translation, and knowledge-injected translation. As introduced in Section III-B, we propose three IR-based methods (IR (CoT), IR (Pseudocode), and IR (NL)) and two knowledge-injected methods (KI (Rules) and KI (RAG)), as well as an iterative refinement strategy. Figure 4 presents the performance of various strategies across different LLMs on the IPL-to-ZKP circuit translation task.

The results reveal a distinct performance hierarchy among the evaluated strategies. KI (Rules) proves most effective, achieving 40.18% CSR, 33.13% PR, and 29.14% CA (127.30% relative gain over Direct). This growth demonstrates that domain-specific expertise aids both syntactic accuracy and semantic consistency. Refinement follows closely, more than doubling baseline CA to 26.39%. KI (RAG) shows divergence: a 13.41% CSR lift but a 16.71% CA drop. Manual inspection of 10% random samples reveals that 80.35% of retrieved examples are irrelevant to the target method, suggesting that noisy retrieval introduces mismatched logical fragments that interfere with constraint construction. IR strategies generally underperform, with IR (Pseudocode) suffering the most severe collapse (-55.15% CA), indicating that IR abstraction weakens source-level cues. Our observations reveal that IR fail to accurately capture source-level information, suffering from two major issues: factual errors and loss of critical information.

This contrasts with prior IPL-to-IPL translation findings [38], [39], where RAG and IR strategies delivered strong performance by supplying useful library mappings and bridging syntactic gaps between IPLs. For ZKP Circuit, however, directly transplanting those experiences proves ineffective, as the fundamental challenge lies not in surface-level adaptation but in reconstructing computation as constraints. This highlights the need for future approaches that leverage the unique characteristics of IPL-to-ZKP Circuit translation.

Finding 4: KI (Rules) and Refinement both yield substantial gains across all metrics, while KI (RAG) and IR strategies cause negative transfer. RAG suffers from irrelevant retrieval, and IR introduces factual errors and information loss. Unlike prior IPL-to-IPL studies where RAG and IR worked well, these strategies prove counterproductive for Circom. This indicates that developers should prioritize domain-specific constraint injection or refinement when building ZKP translation systems.

The efficacy of translation strategies varies sharply with LLMs' capacity. IR (CoT) benefits top-tier LLMs like GPT and Gemini, improving their CA from 17%-24% to 28%-35%, but harms lightweight models such as Llama, which deteriorate to 0%-3%. This is because smaller LLMs lack the reasoning capacity to produce correct intermediate reasoning chains, and their erroneous steps compound into degraded final outputs [40]. KI (RAG) inflates CSR in small LLMs from a 3%-12% baseline to 5%-29%, yet their CA remains stagnant at 0%-5% against a baseline of 1%-10%. This exposes a syntax-logic decoupling where retrieval enables surface mimicry but fails to convey constraint mapping. Refinement and KI (Rule) only work beyond a capacity threshold. Top-tier LLMs reach 48%-65% CA with up to 276% growth, while lightweight models stagnate below 17%, confirming that iterative correction and rule comprehension require sufficient model scale, as smaller models lack the capacity to accurately interpret and apply domain-specific rules during refinement.

Finding 5: Strategy effectiveness in IPL-to-ZKP Circuit translation is strongly gated by model capacity. Top-tier LLMs leverage KI (Rule) and Refinement to reach CA of 48%-65%, while lightweight LLMs stagnate below 17%. KI (RAG) reveals sharp syntax-logic decoupling in small LLMs, inflating CSR without improving CA. Interestingly, IR (CoT) benefits top-tier LLMs but harms lightweight ones, indicating that chain-of-thought reasoning requires sufficient model capacity to avoid error propagation.

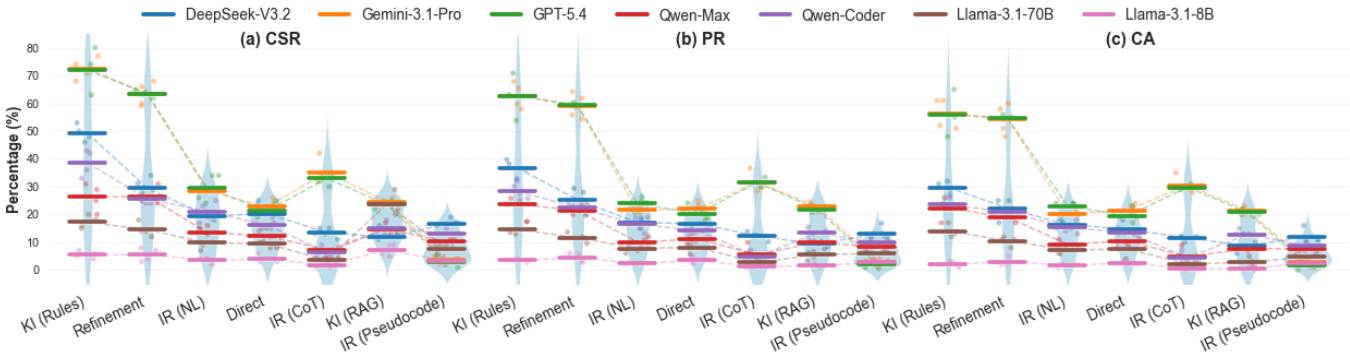

Fig. 4. Performance distribution of seven LLMs across diverse translation strategies

C. RQ3: Security Analysis

Unlike standard IPL-to-IPL translation where functional correctness is the primary concern, security is the top priority in ZKP circuit generation, as any vulnerability can break the entire proof system [41]. A functionally correct circuit can still harbor underconstrained vulnerabilities that enable malicious provers to forge invalid proofs. To assess this risk, we conduct a security evaluation using three mainstream analyzers (ConsCS [31], Picus [33], and ZKFuzz [32]), adopting the Difference in Insecure Rate (DIR) to compare LLMs and translation strategies, with results in Figure 5.

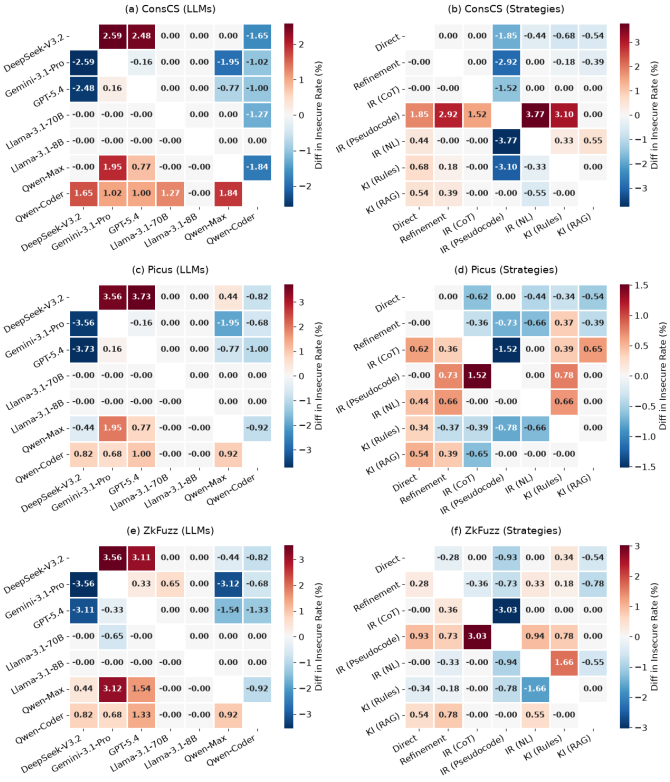

Fig. 5. Security performance comparison using three ZKP analyzers (ConsCS, Picus, ZkFuzz). Values represent DIR = (row insecure rate) - (column insecure rate). Red (positive) means row is less secure than column; blue (negative) means row is more secure. Results shown across LLMs (a, c, e) and translation strategies (b, d, f).

We find that LLMs exhibit significant stratification in DIR across three security testing frameworks. Gemini 3.1-Pro and GPT-5.4 achieve the highest security, with DIR intervals of -3.56%–0.65% and -3.73%–0.16%, respectively, benefiting from their strong reasoning abilities and constraint semantics understanding. Qwen-Max and DeepSeek-V3.2 follow closely, with intervals of -1.84%–3.12% and -1.65%–3.73%. Qwen-Max consistently outperforms Qwen-Coder (-0.00%–1.84%), likely due to its superior general reasoning. The Llama series shows -1.27%–0.00%, but this does not indicate high security. Instead, it reflects limited capability: Llama can only handle basic samples, resulting in a restricted subset where all LLMs perform similarly, which masks the true performance gap.

Finding 6: As a unique perspective of this study on ZKP, security evaluation via ConsCS, Picus, and ZKFuzz reveals significant stratification among LLMs. Gemini 3.1-Pro and GPT-5.4 are most secure (DIR: -3.73%–0.65%), benefiting from strong reasoning and constraint understanding. Qwen-Max and DeepSeek-V3.2 follow (DIR: -1.84%–3.73%). Qwen-Max consistently outperforms its counterpart Qwen-Coder, likely due to stronger general reasoning. Llama shows narrow DIR (-1.27%–0.00%), reflecting limited capability rather than security, as it only handles basic samples where all models perform similarly.

We find substantial variations in DIR across different translation strategies. IR (CoT), IR (NL), and KI (Rules) constitute the top security tier, with DIR intervals of -3.03%–0.65%, -3.77%–1.66%, and -3.10%–0.68%, respectively. IR (CoT) benefits from structured reasoning that preserves constraint semantics throughout the translation process. IR (NL) focuses on high-level program intent, preventing the model from being misled by imperative implementation details that may not map cleanly to constraints. KI (Rules) provides empirical heuristics that directly guide LLMs toward secure constraint patterns. Refinement and Direct fall into the middle tier, with DIR ranges of -2.92%–0.37% and -1.85%–0.34%. KI (RAG) remains close to the baseline, ranging from -0.65% to 0.78%. In contrast, IR (Pseudocode) ranks the least secure with DIR of 0.00%–3.77%, because pseudocode abstraction loses critical semantic details needed for complete constraint construction.

Notably, security performance does not strictly align with logical correctness. IR (CoT) achieves top-tier security without leading in logical accuracy, suggesting a weak correlation be-

tween constraint completeness and logical correctness. Refinement primarily addresses syntax-level issues and superficial compilation errors, lacking the capacity to resolve underlying semantic defects, which limits its overall security performance.

Finding 7: Translation strategies are divided into three security tiers. IR (CoT), IR (NL) and KI (Rules) achieve optimal security with DIR of -3.03%–0.65%, -3.77%–1.66% and -3.10%–0.68%. Refinement and Direct show moderate steady performance at -2.92%–0.37% and -1.85%–0.34%. KI (RAG) remains neutral while IR (Pseudocode) ranks the worst at 0.00%–3.77%. For developers, achieving security requires more than pursuing logical correctness or relying on syntax-level fixes; they should prioritize strategies that enforce constraint-level reasoning, such as IR (CoT) or KI (Rules), to avoid hidden vulnerabilities that superficial revisions cannot address.

D. RQ4: Error Analysis

To gain a deeper understanding of the challenges LLMs face when synthesizing Zero-Knowledge circuits, we conducted an extensive post-mortem analysis of the compilation failures. By analyzing the error logs from the Circom compiler, we identified four predominant error codes: T3001 (Non-quadratic constraints), P1012 (Illegal expressions), P1008 (Syntax/Parsing issues), and T2021 (Undefined symbols). Leveraging SymPy for algebraic verification and regex-based keyword semantic bucketing, we systematically processed all 47,765 failed samples. This taxonomy classifies failures into 9 major categories and 81 subcategories. Table V presents the distribution of primary errors (threshold $\geq$ 1%) across different strategies. For each subcategory, we curated representative code snippets and descriptive metadata, the full version of which is hosted in our repository [13].

Syntax errors in Circom are primarily reported as P1008 ("missing semicolon") and P1012 ("illegal expression"), yet 60.7% of all syntax error lines trigger both codes simultaneously. Manual inspection reveals that most P1008 errors are not caused by genuinely missing semicolons. Instead, when LLMs introduce IPLs constructs, the parser flags it as an illegal expression (P1012) and then reports a spurious missing semicolon (P1008) at the last position it could successfully parse. The dominant subcategory, *Declaration-Initialization Fusion* (11.56%), illustrates this pattern: LLMs write `signal x = expr;`, fusing declaration and assignment into a single unsupported statement. *Illegal Comparison Expression* (4.35%) and *Illegal Index Expression* (3.95%) embed ==, >, or bracket indexing directly into signal assignments, while *Boolean-Ternary Logic Misuse* (1.13%) introduces `?:` operators that have no syntactic role in Circom. *Invalid Type Declaration* (2.48%) reveals that LLMs invent type modifiers such as `signal input long`. *Component Block Declaration Hallucination* (2.06%) and *Component Public Modifier Hallucination* (2.04%) reflect confusion with object-oriented or smart-contract syntax, where LLMs attempt to declare components with block bodies or `public` visibility annotations.

Non-Quadratic Constraints and Algebraic Violation errors correspond to constraint-related defects, collectively making up 13.63% of all failure cases. Circom enforces a strict rank-1 quadratic form for all constraint definitions, yet our algebraic analysis reveals that LLMs frequently and systematically break this core syntactic and semantic restriction. The dominant subcategory, *Parallel Sum (Needs Intermediate Signal)* (4.66%), occurs when LLMs write expressions like `a * b + c * d` in a single constraint rather than decomposing them into one multiplication per line. Closely related is *High-Order Polynomial* (1.29%), where LLMs multiply three or more signals together in one statement, producing constraints of degree 3 or higher. Beyond polynomial violations, we observe frequent use of operators with no algebraic counterpart in the finite-field constraint model: *Forbidden Non-Polynomial Operator* (1.14%) and *Forbidden Division* (1.02%), such as floating-point division, modulo, or ternary expressions, reflect a persistent tendency to treat Circom as an imperative language. A further 2.39% of T3001 errors arise from *Constraint-Only (Missing Assignment)*, where LLMs write a pure constraint (`a === b`) without any signal-driving assignment (`a <== ...`), leaving the signal uninitialized.

Undefined Symbol Calls (T2021, 29.90%) constitute the single largest error category, dominated by hallucinated component names that do not exist in circomlib: LLMs invoke `Mul()` instead of using the native `*` operator, or call `GreaterThan()` and `Div()` rather than importing the correct comparator or division templates.

Dynamic Signal in Static Context errors (10.43%), particularly *Declaration in While Scope* (8.94%) and *Dynamic Index* (1.10%), show that LLMs frequently ignore Circom's requirement that all circuit topology be fixed at compile time, attempting to declare signals inside loops or use signal-dependent array indexing.

Several smaller but persistent categories further illustrate the gap between IPLs and ZKP circuit design: **Duplicate Symbol** (1.84%) occurs when LLMs redefine templates already provided by circomlib; **Wrong Argument Count** (2.31%) reflects providing more or fewer parameters than expected for parameterized templates; and **Undefined Port** (0.54%) reveals confusion over component interfaces.

Finding 8: From 47,765 failed LLM-generated Circom samples, we establish a taxonomy of 9 major failure categories and 81 subcategories. Undefined symbol errors dominate (29.90%), followed by syntax/parse errors (37.58%) and non-quadratic constraint violations (13.63%). The remaining six categories include initialization order, template errors, missing imports, bound violations, control flow issues, and witness mismatches. Three root causes emerge: IPL-style syntax confusion, ignorance of R1CS quadratic rules, and misunderstanding static circuit semantics. Developers can eliminate over 80% of compilation failures by explicitly forbidding IPL-style constructs in prompts, enforcing signal declaration-assignment separation, and providing correct circomlib template names.

Based on the error taxonomy established in our analysis, we modified the Refinement strategy by augmenting compiler error messages with subcategory-specific examples and repair guidance drawn from our repository [13]. For each of the 81 fine-grained error subcategories, we selected one representa-

TABLE V
PRIMARY ERROR DISTRIBUTION BY SUBCATEGORY AND STRATEGY (% , THRESHOLD $\geq 1\%$)

Error Category (Code)	Direct	KI (Rules)	KI (RAG)	IR (NL)	IR (Pse.)	IR (CoT)	Ref.	**Total**
Syntax and Parse Errors	19.55%	26.92%	11.96%	29.30%	73.01%	63.18%	17.39%	37.58%
Declaration-Initialization Fusion (P1008 & P1012)	10.34%	12.29%	5.89%	17.67%	4.57%	25.93%	5.44%	11.56%
Illegal Comparison Expression (P1012)	0.33%	0.16%	0.05%	0.24%	16.35%	6.29%	0.21%	4.35%
Illegal Index Expression (P1012)	0.56%	1.01%	0.18%	0.88%	13.22%	6.01%	0.58%	3.95%
Invalid Type Declaration (P1008 & P1012)	0.03%	0.00%	0.00%	0.06%	11.83%	0.93%	0.06%	2.48%
Component Block Declaration Hallucination (P1008)	0.09%	0.83%	0.54%	0.04%	7.35%	2.53%	0.15%	2.06%
Component Public Modifier Hallucination (P1008)	0.09%	0.83%	0.54%	0.04%	7.35%	2.45%	0.15%	2.04%
Illegal Operator in Expression (P1012)	0.31%	0.79%	0.46%	0.84%	2.06%	3.26%	0.73%	1.31%
Boolean-Ternary Logic Misuse (P1008 & P1012)	2.18%	0.71%	0.28%	1.09%	0.93%	0.24%	2.94%	1.13%
Numeric Suffix Hallucination (P1008 & P1012)	0.45%	0.51%	0.14%	0.37%	1.13%	3.11%	0.56%	0.98%
Illegal Context Declaration (P1012)	0.43%	2.29%	0.12%	0.30%	0.42%	2.66%	0.41%	0.93%
Invalid Signal Type Modifier (P1008)	1.52%	1.17%	0.00%	1.09%	0.29%	0.72%	2.02%	0.88%
Functional Logic Hallucination (P1012)	0.75%	0.95%	0.41%	1.07%	0.94%	0.33%	0.71%	0.72%
Bare Illegal Expression (P1012)	0.49%	0.61%	0.15%	0.54%	0.29%	1.47%	0.77%	0.61%
Undefined Symbol Calls	47.26%	19.23%	52.81%	23.93%	11.83%	12.41%	47.05%	29.90%
Undefined Multiplication Symbol (T2021)	21.89%	2.85%	25.36%	5.52%	2.39%	2.76%	14.53%	10.82%
Undefined Comparison Symbol (T2021)	7.13%	4.70%	10.65%	6.31%	3.87%	3.22%	9.99%	6.37%
Undefined Division/Modulo Symbol (T2021)	8.77%	5.22%	7.89%	6.93%	2.67%	2.45%	13.35%	6.34%
Undefined Addition/Subtraction Symbol (T2021)	5.23%	0.99%	7.08%	2.30%	1.39%	1.65%	6.83%	3.58%
Undeclared Symbol (T2021)	3.93%	5.02%	0.66%	2.32%	1.13%	1.62%	1.61%	2.18%
Non-Quadratic Constraints	16.86%	12.23%	12.57%	22.13%	8.71%	9.91%	17.37%	13.63%
Parallel Sum (Needs Intermediate Signal) (T3001)	5.78%	4.88%	4.54%	7.30%	3.44%	2.72%	5.33%	4.66%
Constraint-Only (Missing Assignment) (T3001)	2.19%	0.45%	2.19%	2.57%	1.97%	4.26%	2.62%	2.39%
High-Order Polynomial (T3001)	1.98%	1.42%	1.70%	1.74%	0.65%	0.33%	1.70%	1.29%
Forbidden Non-Polynomial Operator (T3001)	2.19%	1.01%	0.92%	2.68%	0.28%	0.20%	1.44%	1.14%
Forbidden Division (T3001)	1.47%	0.93%	0.73%	2.27%	0.63%	0.14%	1.55%	1.02%
Topological Sequence Violation (T3001)	0.83%	0.47%	0.65%	0.94%	0.44%	1.32%	0.95%	0.79%
Forbidden Bitwise Operation (T3001)	0.88%	0.38%	0.39%	1.61%	0.24%	0.13%	0.71%	0.57%
Dynamic Signal in Static Context	10.51%	28.28%	10.57%	15.20%	1.58%	6.30%	9.91%	10.43%
Declaration in While Scope (T2011)	8.24%	26.96%	8.69%	13.71%	1.52%	5.78%	5.97%	8.94%
Dynamic Index (T20462)	1.71%	1.15%	0.82%	0.71%	0.00%	0.47%	3.83%	1.10%
Project/Module Errors	1.15%	2.43%	3.42%	2.85%	1.86%	4.95%	0.56%	2.52%
Duplicate Symbol (T2008)	0.88%	1.28%	1.82%	2.36%	1.52%	4.18%	0.32%	1.84%
Wrong Argument Count (T2012)	3.09%	3.08%	3.60%	2.66%	1.02%	1.28%	2.11%	2.31%
Logic Errors	0.88%	6.09%	2.47%	2.42%	0.74%	0.86%	3.27%	2.09%
Type Mismatch	0.32%	1.56%	0.93%	1.14%	0.69%	0.23%	1.87%	0.87%
Array Access Mismatch (T2032)	0.12%	0.22%	0.04%	0.75%	0.01%	0.05%	1.07%	0.26%
Component Initialization/Interface	0.37%	0.18%	1.66%	0.37%	0.57%	0.88%	0.45%	0.68%
Undefined Port (T2046)	0.37%	0.18%	1.01%	0.37%	0.49%	0.88%	0.24%	0.54%

Note: Only subcategories with a proportion $\geq 1\%$ in at least one strategy or the total column are shown. Minor categories (all strategies and total $< 1\%$) are omitted from this table. The complete distribution is available in our repository [13].

TABLE VI
PERFORMANCE COMPARISON ACROSS STRATEGIES (%)

Model	Direct			Refinement			KI (Rules)			Refinement (Knowledge)		
	CSR	PR	CA	CSR	PR	CA	CSR	PR	CA	CSR	PR	CA
DeepSeek	21.47	18.65	16.18	30.59	27.18	23.24	**51.76**	**40.88**	32.94	44.12	38.12	**33.53**
Gemini	22.35	22.18	21.47	65.00	62.24	55.88	**72.94**	**65.35**	**58.24**	67.35	63.94	57.35
GPT	20.29	19.82	18.53	65.29	62.94	56.47	**71.47**	63.47	56.18	67.65	**64.88**	**57.65**
Llama-70B	10.88	9.76	8.82	16.18	13.82	12.35	20.00	17.35	16.47	**23.82**	**20.12**	**18.24**
Llama-8B	4.71	4.18	2.94	6.47	5.41	3.53	6.47	4.18	2.65	**10.59**	**7.18**	**4.41**
Qwen3-Max	11.18	10.94	10.29	26.18	22.18	19.12	29.12	27.00	**25.00**	**32.06**	**28.41**	24.71
Qwen-Coder	16.76	15.88	15.00	27.35	25.35	23.53	**40.88**	31.71	26.47	35.88	**32.82**	**30.59**

tive code snippet and wrote a diagnostic description with a suggested fix. From the original 100 sample test set for each model strategy pair, we reserved 15 samples for constructing these exemplars and evaluated on the remaining 85. Table VI reports the results of this augmented approach, denoted as Refinement (Knowledge).

The augmented Refinement strategy yields consistent and substantial improvements over the base Refinement across nearly all LLMs and metrics. CSR improves from 6.47%-65.29% to 10.59%-67.65%, PR from 5.41%-62.94% to 7.18%-64.88%, and CA from 3.53%-56.47% to 4.41-57.65%. Performance gains are uneven across model tiers. Top LLMs such as GPT and Gemini improve by only $\sim$ 2%, while weaker LLMs benefit substantially more. This indicates that the error taxonomy is most valuable where LLMs lack intrinsic knowledge of Circom, narrowing the gap between weaker and stronger LLMs. More importantly, Refinement (Knowledge) performs competitively with KI (Rules), the strategy that relies on manually curated knowledge injection. For four out of seven LLMs, the augmented Refinement actually surpasses KI (Rules) on CSR and PR, while for the remaining three

the two strategies are within a narrow margin. KI (Rules) requires domain experts to anticipate failure modes and codify preventive rules before generation, Refinement (Knowledge) derives its guidance directly from an empirical error taxonomy built by analyzing model outputs after generation, offering a scalable and data driven alternative for improving LLM based circuit synthesis.

Finding 9: Our taxonomy of 9 prevalent failure categories provides actionable lessons for practitioners and researchers. Data driven error aware refinement derived from our benchmark substantially improves LLM generated Circom, achieving effectiveness comparable to KI (Rules). For developers, systematically understanding failure modes in ZK circuit synthesis directly enhances LLM correctness and security, offering a scalable blueprint that distinguishes our work from general purpose code translation approaches.

V. Threats to Validity

Threats to external validity relate to the generality of our findings. Our evaluation focuses on translating IPLs to Circom, the most widely adopted ZKP language, which offers a low-level constraint representation close to the underlying R1CS model. While this makes Circom representative, results might not fully generalize to higher-level languages like Noir. Nevertheless, the core challenges we identify, such as reconstructing computation as constraints and avoiding underconstrained logic, are inherent to ZKP circuit generation regardless of the target language. Additionally, our taxonomy was derived from 47,765 error samples across seven state-of-the-art LLMs, ensuring representative categorization of error patterns. We therefore consider the threats to external validity minimal. **Threats to internal validity** involve potential data leakage and manual analysis subjectivity. We mitigated leakage by manually constructing TransZKEval, ensuring source target pairs and test suites were not in pre training corpora. Regarding qualitative analysis, mapping compiler diagnostics to subcategories could be biased. We minimized this using a pipeline with SymPy for algebraic verification and keyword based bucketing. **Threats to construct validity** concern the metric comprehensiveness. We adopted CSR for syntactic adherence, PR for functional verification, and CA for overall success. These provide a better view than BLEU or CodeBLEU, which correlate poorly with circuit correctness. Using execution based validation through the Circom compiler ensures our correctness assessment is grounded in actual provability. For RQ3, although the evaluation tools (ConsCS, Picus, ZkFuzz) exhibit high Unknown rates, we focus on relative performance across different LLMs using a unified set of industry standard tools, so the comparative impact of this threat is minimal.

VI. Related Work

A. Zero Knowledge Proofs

Recent advances in zero knowledge proofs have spurred research across multiple directions. For circuit development, Circom [6] provides a domain specific language for designing arithmetic circuits compiled to R1CS. For security verification, tools such as ConsCS [31] and Picus [33] automatically verify Circom circuits for underconstrained vulnerabilities and constraint determinism. For testing, MTZK [42] employs metamorphic testing to uncover compilation errors in ZK compilers, fAmulet [43] detects finalization failure bugs in zkRollup protocols, and SNARKProbe [44] focuses on revealing bugs in zkSNARK implementations. More recently, researchers have explored LLMs for ZK code generation; ZK-Eval [45] systematically evaluates LLMs on Circom and Noir generation, revealing that LLMs excel at surface level syntax but struggle with gadget level reasoning and semantic correctness. However, all existing work focuses on either generating circuits from specifications or analyzing existing circuits. The task of translating IPLs into ZK circuits remains unaddressed. Our TransZKEval fills this gap by providing a benchmark for translation IPLs to Circom.

B. Automated Code Translation

Automated code translation has evolved from rule based techniques [46]–[48] to approaches driven by LLMs [23], [49]. Early benchmarks such as G-TransEval [50] and CodeTransOcean [9] evaluate function level translation within IPLs. Subsequent work has expanded to class-level [8], and repository-level [51]–[53]. Meanwhile, domain specific benchmarks have emerged for third-party library translation [24], smart contract translation [54], high-performance computing [55] and so on. However, none of these efforts tackle the translation from IPLs to the constraint based PL of ZK circuits. Our TransZKEval benchmark fills this void by assessing the translation of IPL into Circom, providing a benchmark for circuit translation.

VII. Conclusion and Future Work

This work constructs *TransZKEval*, the first benchmark dedicated to evaluating code translation from IPLs to ZKP circuits. We design six representative translation strategies and conduct extensive experiments on 7 LLMs across commercial, general, and code-specialized families. Our evaluation reveals severe performance degradation compared to conventional code translation, identifies the efficacy and limitations of different translation strategies, and exposes critical security risks including underconstrained vulnerabilities. We further present a fine-grained error taxonomy that characterizes the dominant failure modes in LLM-generated ZKP circuits, providing unique insights for the community.

Based on our findings, our future work aims to design a dedicated translation method tailored for ZKP circuit generation to address the paradigm gap between IPL and arithmetic constraints. We will incorporate constraint-aware reasoning, domain-specific structure priors, and automated validation into a unified pipeline to improve both correctness and security. Additionally, we plan to extend *TransZKEval* with more complex real-world ZKP tasks and support additional circuit languages to broaden its impact and usability.

Data Availability: We open-source the benchmark, evaluation pipeline, and all experimental results of *TransZKEval* at [13].

REFERENCES

[1] O. Kuznetsov, Y. Kuznetsova, G. Ziyatbekova, Y. Kovalenko, and R. Palahusynets, "Performance evaluation of zk-snark protocols for privacy-preserving sensor data verification: A systematic benchmarking study," *Sensors (Basel, Switzerland)*, vol. 26, no. 8, p. 2486, 2026.

[2] Ethereum Foundation, "Zero-knowledge rollups," Ethereum development documentation, 2026, accessed: 2026-05-08. [Online]. Available: https://ethereum.org/developers/docs/scaling/zk-rollups/

[3] Uniswap Community, "Uniswap v4 sdk rust," crates.io, 2025. [Online]. Available: https://crates.io/crates/uniswap-v4-sdk

[4] Ethereum DeFi Community, "web3-ethereum-defi (eth-defi)," PyPI, 2025. [Online]. Available: https://pypi.org/project/web3-ethereum-defi/

[5] Drift Labs, "Drift protocol sdks," Drift Protocol Documentation, 2025. [Online]. Available: https://docs.drift.trade/developers

[6] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, "Circom: A circuit description language for building zero-knowledge applications," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4751, 2022.

[7] J. Liu, I. Kretz, H. Liu, B. Tan, J. Wang, Y. Sun, L. Pearson, A. Miltner, I. Dillig, and Y. Feng, "Certifying zero-knowledge circuits with refinement types," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 1741–1759.

[8] P. Xue, L. Wu, Z. Yang, C. Wang, X. Li, Y. Zhang, J. Li, R. Jin, Y. Pei, Z. Shen *et al.*, "Classeval-t: Evaluating large language models in class-level code translation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1421–1444, 2025.

[9] W. Yan, Y. Tian, Y. Li, Q. Chen, and W. Wang, "Codetransocean: A comprehensive multilingual benchmark for code translation," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 5067–5089.

[10] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen, and X. Peng, "No more manual tests? evaluating and improving chatgpt for unit test generation," *arXiv preprint arXiv:2305.04207*, 2023.

[11] W. Wang, C. Yang, Z. Wang, Y. Huang, Z. Chu, D. Song, L. Zhang, A. R. Chen, and L. Ma, "Testeval: Benchmarking large language models for test case generation," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 3547–3562.

[12] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2023.

[13] A. Authors, "Transzkeval: Unraveling llm behaviors in zkp circuit translation," https://github.com/AnonymousAuthor-123/TransZKEval, 2026, accessed 2026-06-29.

[14] G. van Rossum, B. Warsaw, and A. Coghlan, "Style guide for Python code," PEP 8, 2001, accessed: 2026-05-08. [Online]. Available: https://www.python.org/dev/peps/pep-0008/

[15] Google, "Google java style guide," Google Style Guides, 2023, accessed: 2026-05-08. [Online]. Available: https://google.github.io/styleguide/javaguide.html

[16] V. Veloso and A. Hora, "Characterizing high-quality test methods: a first empirical study," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 265–269.

[17] OpenAI, "Chatgpt," https://chatgpt.com/, accessed: 2026-05-08.

[18] Google, "Gemini," https://gemini.google.com/, accessed: 2026-05-08.

[19] Qwen Team, "Qwen3-Max: Bigger is better," https://qwen.ai/blog?id=qwen3-max, September 2025, accessed: 2026-05-08.

[20] DeepSeek-AI, "DeepSeek V3.2 official release: Strengthened agent capabilities, integrated reasoning," https://api-docs.deepseek.com/zh-cn/news/news251201, December 2025, accessed: 2026-05-08.

[21] Qwen Team, "Qwen3-Coder: Agentic coding in the world," https://qwen.ai/blog?id=qwen3-coder, July 2025, accessed: 2026-05-08.

[22] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[23] Z. Yang, F. Liu, Z. Yu, J. W. Keung, J. Li, S. Liu, Y. Hong, X. Ma, Z. Jin, and G. Li, "Exploring and unleashing the power of large language models in automated code translation," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1585–1608, 2024.

[24] P. Xue, K. Zheng, Z. Yang, Y. Pei, L. Wu, J. Dong, X. Luo, Y. Xiao, F. Liu, Y. Zhang *et al.*, "Translibeval: Demystify large language models' capability in third-party library-targeted code translation," *arXiv preprint arXiv:2509.12087*, 2025.

[25] R. Pan, A. R. Ibrahimzada, R. Krishna, D. Sankar, L. P. Wassi, M. Merler, B. Sobolev, R. Pavuluri, S. Sinha, and R. Jabbarvand, "Lost in translation: A study of bugs introduced by large language models while translating code," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[26] B. Roziere, J. Zhang, F. Charton, M. Harman, G. Synnaeve, and G. Lample, "Leveraging automated unit tests for unsupervised code translation," in *International Conference on Learning Representations*, 2021.

[27] B. Roziere, M.-A. Lachaux, L. Chanussot, and G. Lample, "Unsupervised translation of programming languages," *Advances in Neural Information Processing Systems*, vol. 33, pp. 20 601–20 611, 2020.

[28] DeepSeek, "Deepseek official website," 2026, accessed May 8, 2026. [Online]. Available: https://www.deepseek.com/

[29] Alibaba Cloud, "Alibaba cloud official website," 2026, accessed May 8, 2026. [Online]. Available: https://www.aliyun.com/

[30] NVIDIA Corporation, "Nvidia pretrained ai models," 2026, accessed May 8, 2026. [Online]. Available: https://developer.nvidia.com/ai-models

[31] J. Jiang, X. Peng, J. Chu, and X. Luo, "Conscs: Effective and efficient verification of circom circuits," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, 2025, pp. 616–628.

[32] H. Takahashi, J. Kim, S. Jana, and J. Yang, "zkfuzz: Foundation and framework for effective fuzzing of zero-knowledge circuits," *arXiv preprint arXiv:2504.11961*, 2025.

[33] S. Pailoor, Y. Chen, F. Wang, C. Rodríguez, J. Van Geffen, J. Morton, M. Chu, B. Gu, Y. Feng, and I. Dillig, "Automated detection of under-constrained circuits in zero-knowledge proofs," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1510–1532, 2023.

[34] F. Tip, J. Bell, and M. Schäfer, "Llmorpheus: Mutation testing using large language models," *IEEE Transactions on Software Engineering*, 2025.

[35] Q. Tao, T. Yu, X. Gu, and B. Shen, "Unraveling the potential of large language models in code translation: How far are we?" 2024. [Online]. Available: https://arxiv.org/abs/2410.09812

[36] P. Jaccard, "Nouvelles recherches sur la distribution florale," *Bull. Soc. Vaud. Sci. Nat.*, vol. 44, pp. 223–270, 1908.

[37] G. G. Chowdhury, *Introduction to modern information retrieval*. Facet publishing, 2010.

[38] C.-e. A. Tai, P. Nie, L. Golab, and A. Wong, "Nl in the middle: Code translation with llms and intermediate representations," in *2025 IEEE International Conference on Collaborative Advances in Software and COmputiNg (CASCON)*. IEEE, 2025, pp. 295–300.

[39] M. Bhattarai, M. Vu, J. E. Santos, I. Boureima, and D. O'Malley, "Enhancing cross-language code translation via task-specific embedding alignment in retrieval-augmented generation," in *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing*, 2025, pp. 107–117.

[40] R. Luo, J. Li, C. Huang, and W. Lu, "Through the valley: Path to effective long cot training for small language models," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 4972–4992.

[41] S. Chaliasos, I. Al-Fath, and A. Donaldson, "Towards fuzzing zero-knowledge proof circuits (short paper)," in *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2025, pp. 98–104.

[42] D. Xiao, Z. Liu, Y. Peng, and S. Wang, "Mtzk: Testing and exploring bugs in zero-knowledge (zk) compilers." in *NDSS*, 2025.

[43] Z. Li, X. Peng, Z. He, X. Luo, and T. Chen, "famulet: Finding finalization failure bugs in polygon zkrollup," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 971–985.

[44] Y. Fan, Y. Xu, and C. Garman, "Snarkprobe: An automated security analysis framework for zksnark implementations," in *International Conference on Applied Cryptography and Network Security*. Springer, 2024, pp. 340–372.

[45] Z. Xue, P. Ma, Z. Wang, Y. Zhou, X. Zhang, S. Wang, and J. Rahmel, "From evaluation to enhancement: Large language models for zero-knowledge proof code generation," *arXiv preprint arXiv:2509.11708*, 2025.

[46] Y. Takefuji and M. Dowell, "Novel approach to a rule-based general purpose program translator using paramodulation," *Knowledge-Based Systems*, vol. 1, no. 2, pp. 90–93, 1988.

[47] J. C. Freytag and N. Goodman, "Rule-based transformation of relational queries into iterative programs," in *Proceedings of the 1986 ACM SIGMOD international conference on Management of data*, 1986, pp. 206–214.

[48] A. Corradini, F. L. Dotti, L. Foss, and L. Ribeiro, "Translating java code to graph transformation systems," in *International Conference on Graph Transformation*. Springer, 2004, pp. 383–398.

[49] S. Bhatia, J. Qiu, N. Hasabnis, S. A. Seshia, and A. Cheung, "Verified code transpilation with llms," *Advances in Neural Information Processing Systems*, vol. 37, pp. 41 394–41 424, 2024.

[50] M. Jiao, T. Yu, X. Li, G. Qiu, X. Gu, and B. Shen, "On the evaluation of neural code translation: Taxonomy and benchmark," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 1529–1541.

[51] Y. Wang, Y. Wang, S. Wang, D. Guo, J. Chen, J. Grundy, X. Liu, Y. Ma, M. Mao, H. Zhang *et al.*, "Repotransbench: A real-world multilingual benchmark for repository-level code translation," *IEEE Transactions on Software Engineering*, 2025.

[52] X. Zhang, J. Wen, F. Yang, P. Zhao, Y. Kang, J. Wang, M. Wang, Y. Huang, E. Nallipogu, Q. Lin *et al.*, "Skeleton-guided-translation: A benchmarking framework for code repository translation with fine-grained quality evaluation," *arXiv preprint arXiv:2501.16050*, 2025.

[53] Z. Guan, X. Yin, Z. Peng, and C. Ni, "Repotransagent: Multi-agent llm framework for repository-aware code translation," *arXiv preprint arXiv:2508.17720*, 2025.

[54] R. Karanjai, L. Xudagger, and W. Shi, "Solmover: Feasibility of using llms for translating smart contracts," in *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2024, pp. 1–3.

[55] J. H. Davis, D. Nichols, I. Khillan, and A. Bhatele, "Pareval-repo: A benchmark suite for evaluating llms with repository-level hpc translation tasks," in *Proceedings of the 54th International Conference on Parallel Processing*, 2025, pp. 94–103.